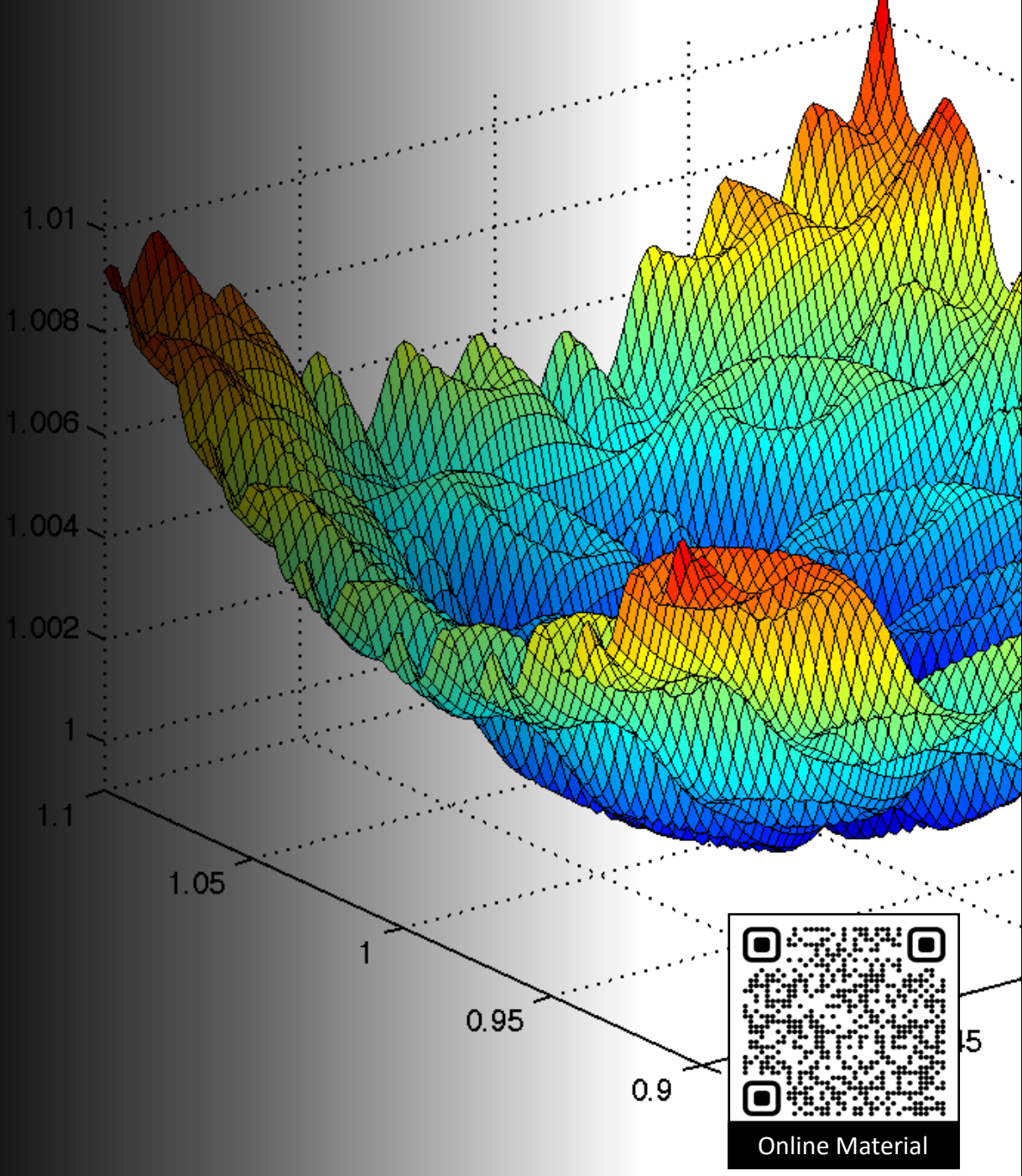
CS 5/7320
Artificial Intelligence

Local Search
AIMA Chapters 4.1 & 4.2

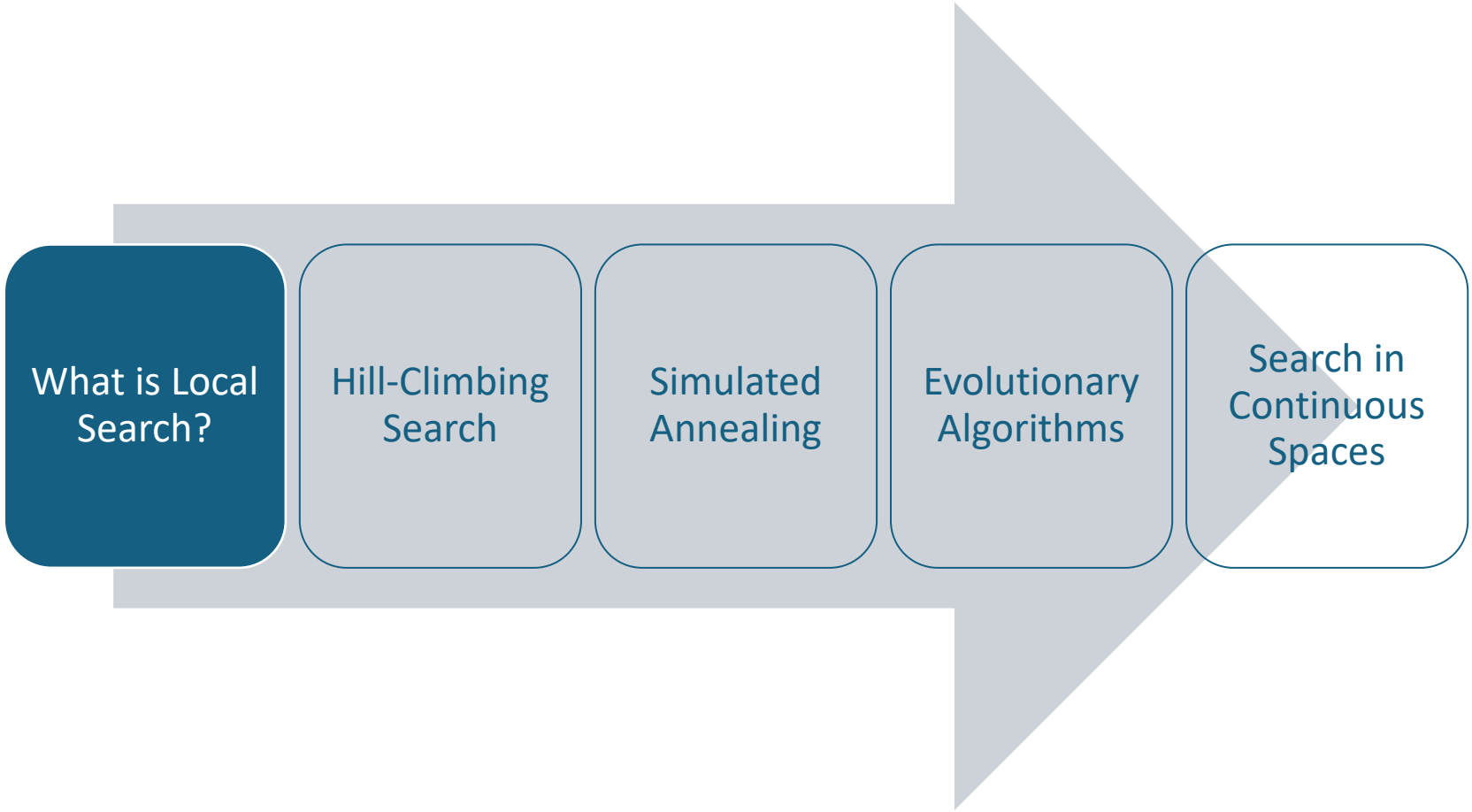
Slides by Michael Hahsler
based on slides by Svetlana Lazepnik
with figures from the AIMA textbook.



Online Material

This work is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.

Contents


What is Local Search?
Hill-Climbing Search
Simulated Annealing
Evolutionary Algorithms
Search in Continuous Spaces

Recap: Uninformed and Informed Search

Tries to **plan** the

best path

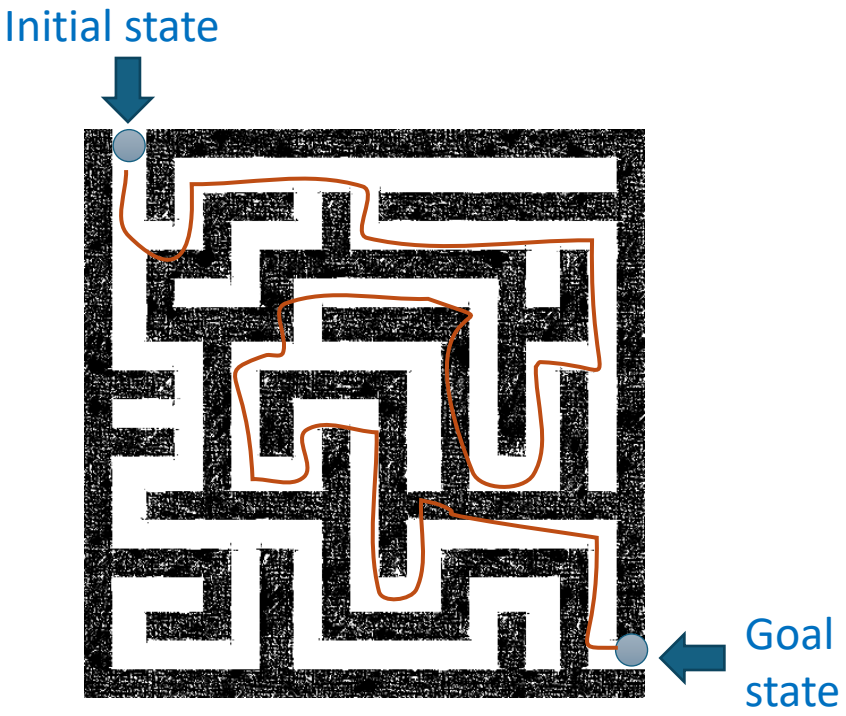
from a

given initial state

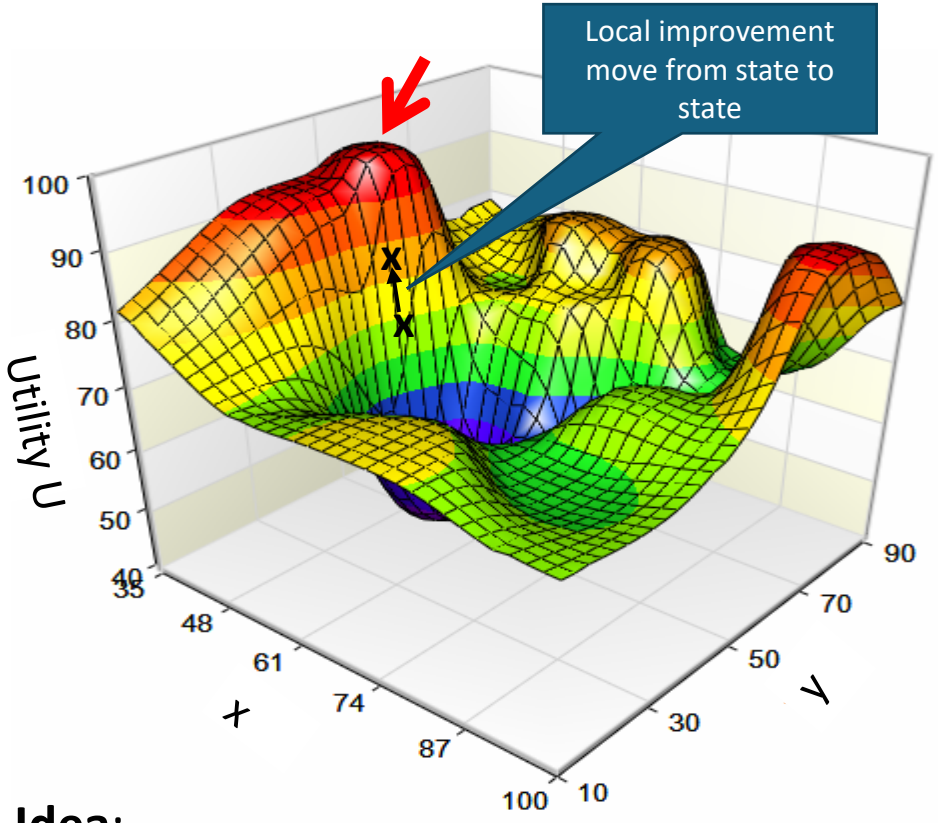
to a

given goal state.

- Often comes with completeness and optimality guarantees (BFS, A* Search, IDS).
- **Issue**: Typically have to search a large portion of the search space and therefore need a lot of time and memory.



Local Search



- Assume we know the utility of each possible state given by a utility function
$$U = u(s)$$
- We use a factored state description. Here $s = (x, y)$
- How can we identify the best or at least a "good" state?
- This is the **optimization problem**:
$$s^* = \underset{s \in S}{\operatorname{argmax}}\, u(s)$$
- We need a fast and memory-efficient way to find the best/a good state.

Idea:

Start with a current solution (a state) and improve the solution by moving from the current state to a "neighboring" better state (a.k.a. performing a series of **local moves**).

Main Differences to Tree Search

a) **The goal states are unknown,** but we know or can calculate the utility for each state. We want to identify a high-utility state.

b) Often, no explicit initial state is given, and the **path to the goal and the path cost are not important.**

c) **No search tree**. Just stores the current state and moves to a "better" state if possible. Needs little memory and only requires very simple code.

Use of Local Search in AI

- **Goal-based agent**: Identify a good goal state with a good utility before planning a path to that state using a planning agent.

- **Utility-based agent**: Always move to a neighboring state with higher utility. A simple greedy method used for
  - complicated/large state spaces or
  - online search.

- **General optimization**: $u(s)$ can be replaced by a general objective function. Local search is an effective heuristic to find good solutions in complicated search spaces.
  E.g., stochastic gradient descent to train neural networks (minimize the prediction error)

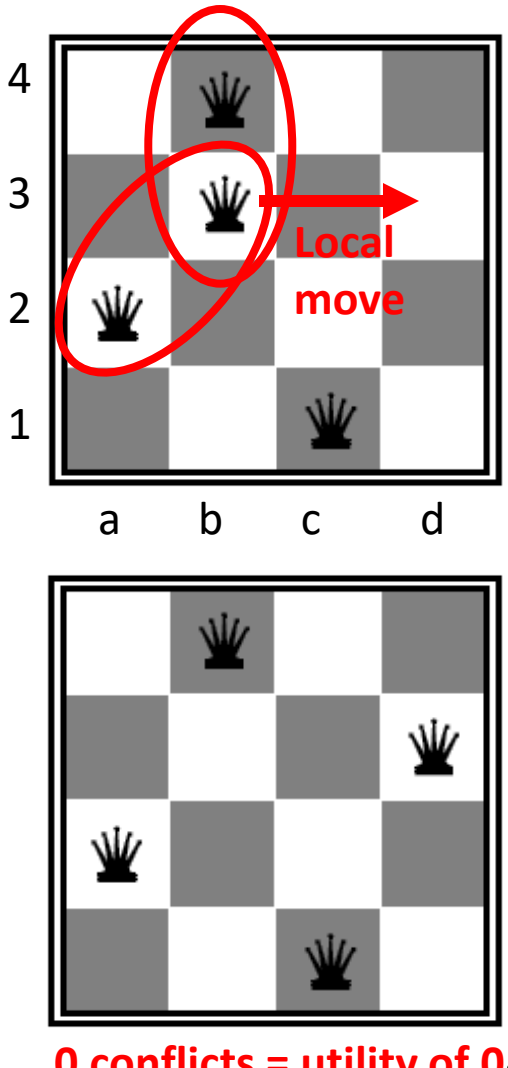
Defining A Local Search Problem

- **State space:** How large is the state space?
- **State representation:** How do we define a factored state representation?
- **Objective function**: What is a possible utility function given the state representation?
- **Local neighborhood:** What states are close to each other?

Replaces the goal state

Replaces the transition function.

Example: *n*-Queens Problem



Place n queens on a $n \times n$ chess board so no two queens are in the same row, column or diagonal.

Defining the search problem:

State space: All possible *n*-queen configurations. How many are there?

- 4-queens problem: $\binom{16}{4} = 1820$

State representation: How do we define a factored representation?

- E.g. $(a2, b3, b4, c1)$

Objective function: What is a possible utility function given the state representation?

- Maximizing utility means minimize the number of pairwise conflicts based on the state representation.

Local neighborhood: What states are close to each other?

- Move a single queen.

Has its optimum at the goal state. Similar to a heuristic in A* search. No need to be admissible.



Example: Traveling salesman problem

Find the shortest tour connecting a given set of cities

- **State space:** all possible tours (states are not individual cities!)
- **State representation:** Order of cities in the tour.
- **Objective function:** minimize the length of the tour.
- **Local neighborhood:** Change the order of visiting a few cities.

Note: We have solved a **different** problem with uninformed/informed search! Each city was defined as a state and the path was the solution.



Hill-Climbing Search aka Greedy Local Search

Idea: keep a single “current” state and try to find better neighboring states.

MotorCycleUSA.com

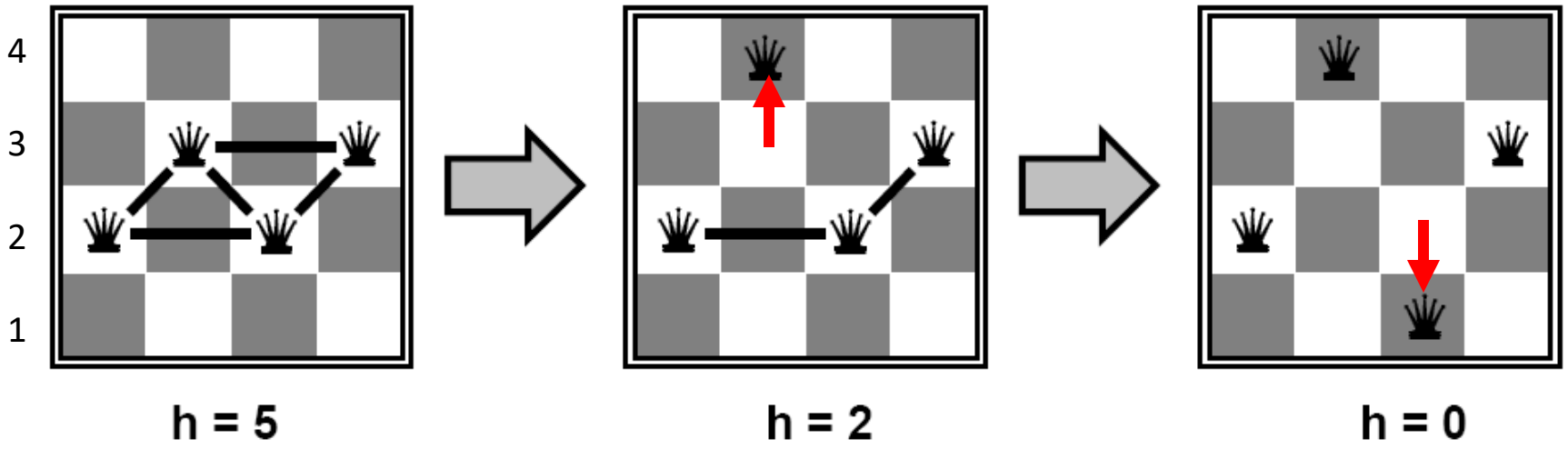
Example: *n*-Queens Problem

- **Goal:** Put *n* queens on an *n* × *n* board with no two queens on the same row, column, or diagonal.
- **State space:** all possible *n*-queen configurations. We can restrict the state space: Only one queen per column.
- **State representation:** row position of each queen in its column (e.g., 2, 3, 2, 3)
- **Objective function:** minimize the number of pairwise conflicts.
- **Local neighborhood:** Move one queen anywhere in its column.

4-queens problem: State space is reduced from 1820 to $4^4 = 256$

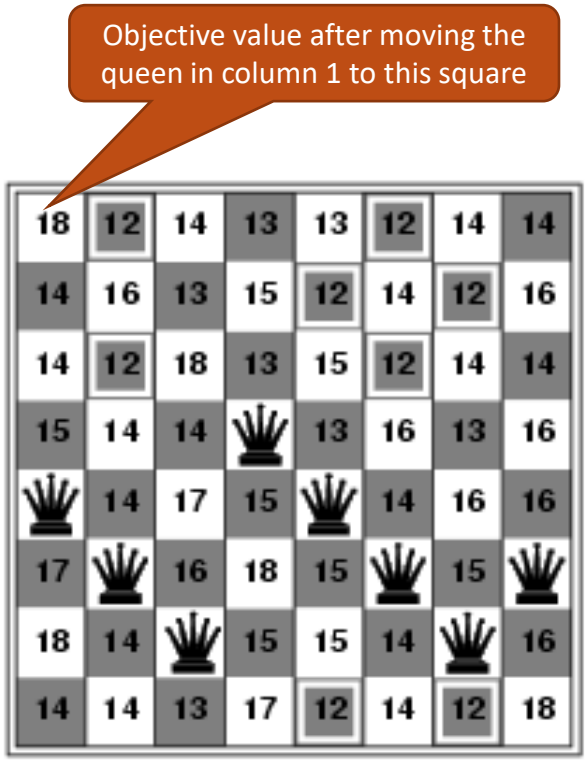
Improvement strategy

- Find a local neighboring state (move one queen within its column) to reduce conflicts.



Example: *n*-Queens Problem 2

Find the best local move: **Evaluate the objective function for all local neighbors** (moving a single queen in its column while leaving the others in place).



Current objective value: $h = 17$

Best local improvement has $h = 12$

Notes:

- There are many options with $h = 12$. We typically pick one randomly.
- **Calculating all the objective values may be expensive!**

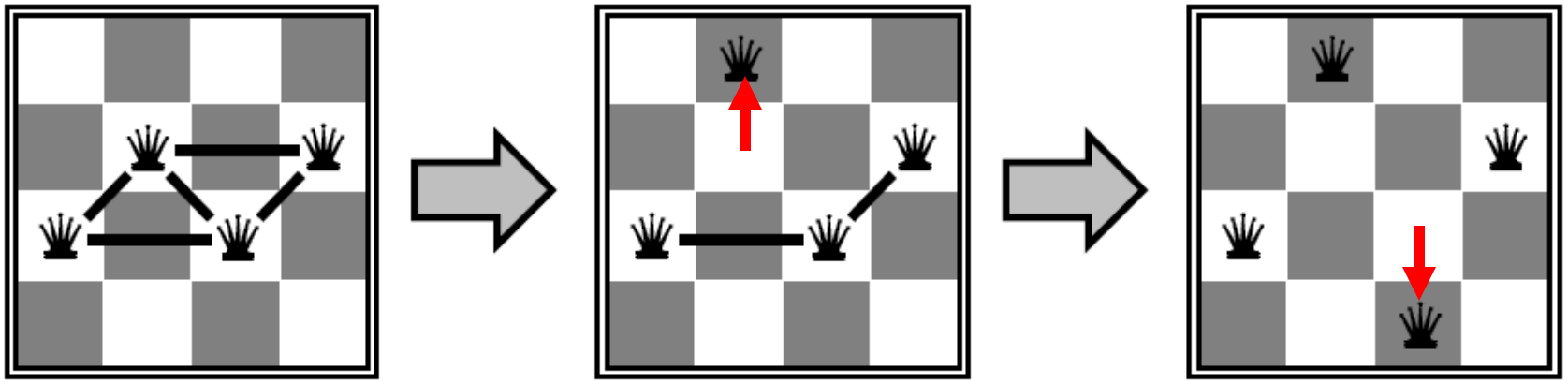
Example: *n*-Queens Problem 3

Formulation as an optimization problem:
Find the best state s^* representing an arrangement of queens.

$$s^* = \text{argmax}_{s \in S} \left(-\text{conflicts}(s)\right)$$

subject to: s has one queen per column

Remember: This makes the problem a lot easier.

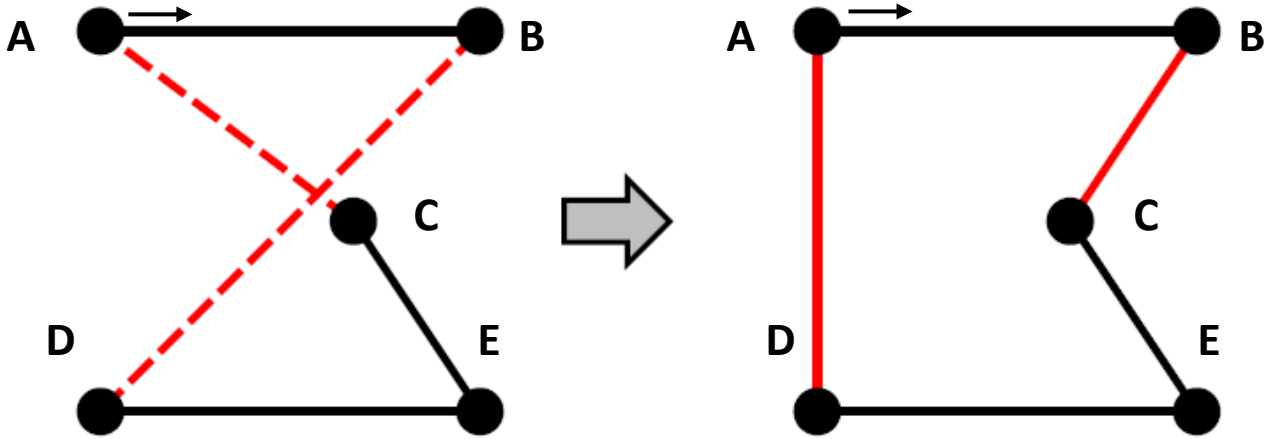


Example: Traveling Salesman Problem 2



- **Goal:** Find the shortest tour connecting n cities
- **State space:** all possible tours
- **State representation:** tour (order in which to visit the cities) = a permutation. There are $n!$ Many permutations.
- **Objective function:** length of tour
- **Local neighborhood:** reverse the order of visiting a few cities

Local move to reverse the order of cities C, E and D:



State representation (permutation): **ABDEC** **ABCED**

Example: Traveling Salesman Problem 3

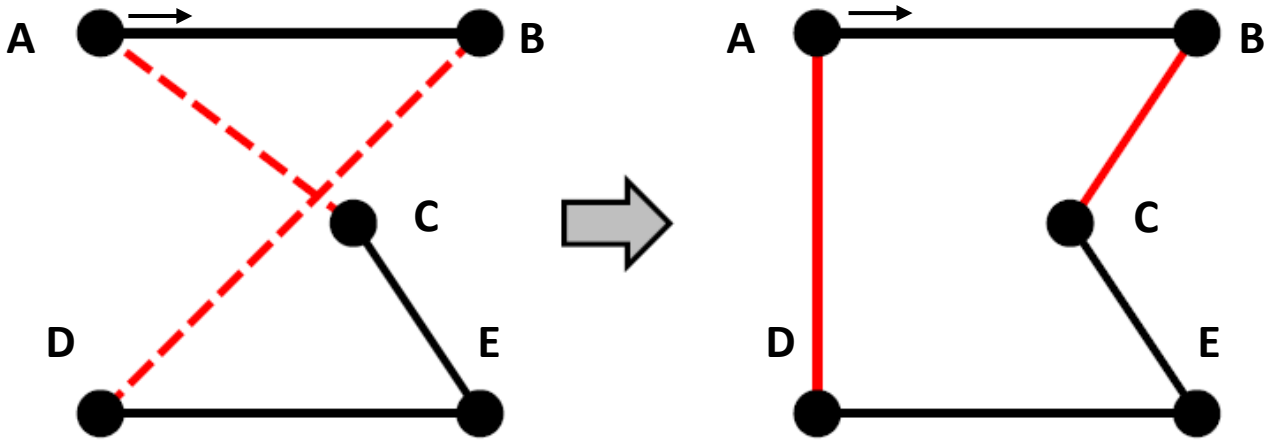


Formulation as an optimization problem:
Find the best tour π

$$\pi^* = \text{argmin}_{\pi \in \Pi} \ \text{tourLength}(\pi)$$

s.t. π is a valid permutation (i.e., sub-tour elimination)

Local move to reverse the order of cities C, E and D:



State representation: **ABDEC** **ABCED**

Hill-Climbing Search (Greedy Local Search)

Maximization

```
function HILL-CLIMBING(problem) returns a state that is a local maximum
    current ← problem.INITIAL
    while true do
        neighbor ← a highest-valued successor state of current
        if VALUE(neighbor) ≤ VALUE(current) then return current
        current ← neighbor
```

We often start with a random state

Use lowest-valued successor and ≥ for minimization

Variants (define what successor to choose)

a. **Steepest ascent hill climbing:** Check all possible successors and choose the highest-valued successor.

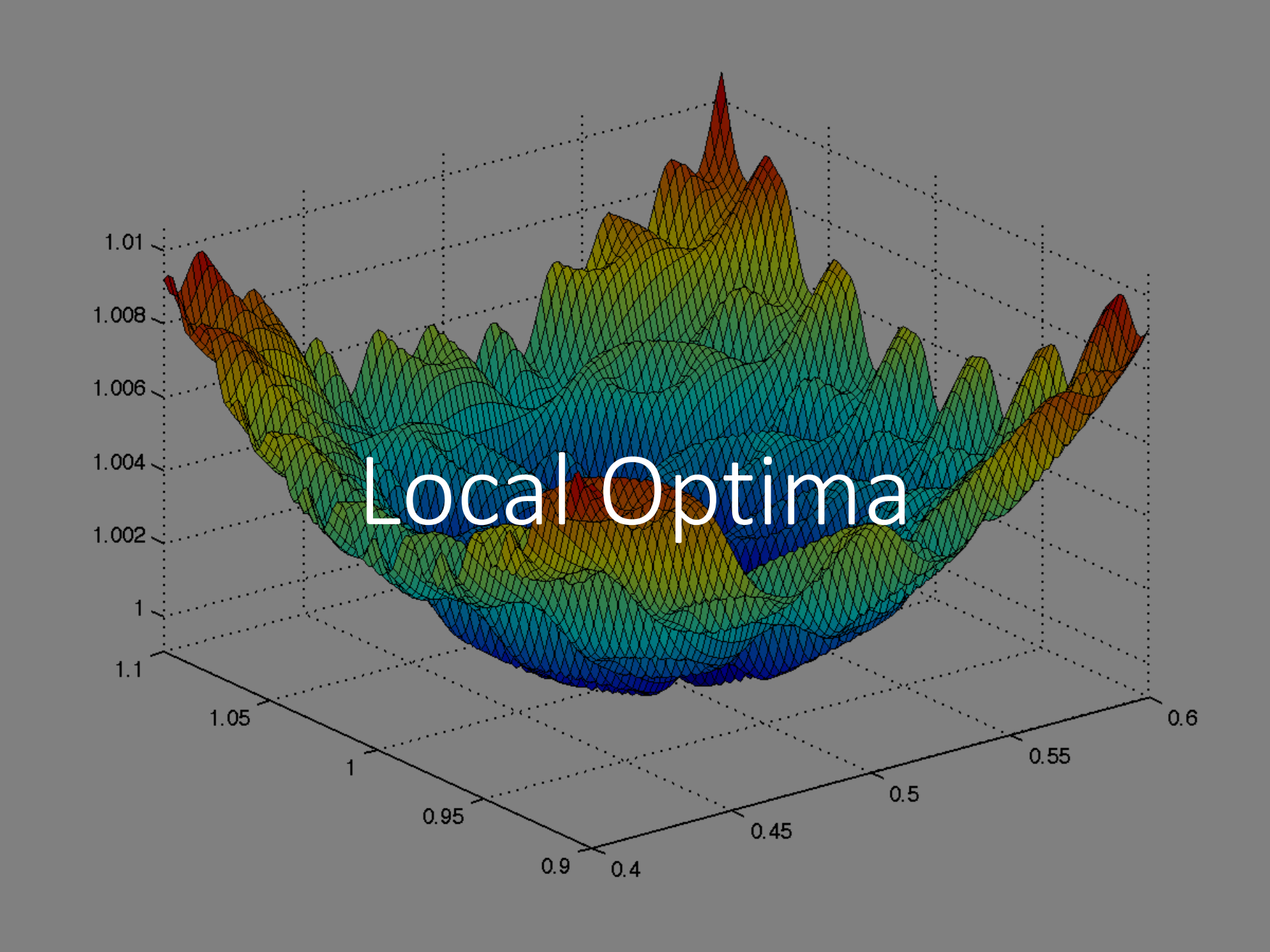
b. **Stochastic hill climbing:** Choose randomly among **all** uphill (improvement) moves.

c. **First-choice stochastic hill climbing**: Generate randomly **one** new successor at a time and only move to better ones. This is what people often mean by "stochastic hill climbing." It is equivalent to b, but computationally much cheaper (does not evaluate h for each neighbor).

Minimization



$h = 17$

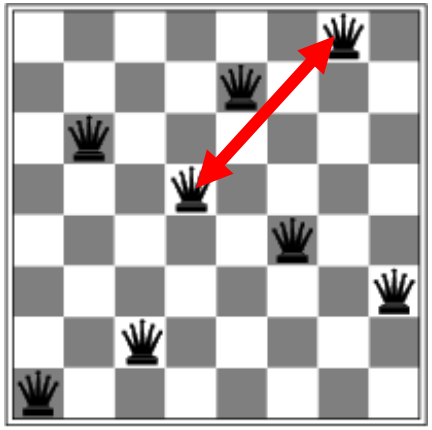

Local Optima
1.01
1.008
1.006
1.004
1.002
1
1.1
1.05
1
0.95
0.9
0.4
0.45
0.5
0.55
0.6

Local Optima

Hill-climbing search is **greedy** (like greedy best-first search) with the objective function as a typically not admissible heuristic.
It only stores the current state (has no frontier data structure) and **just stops at a dead end**.

Is it complete/optimal?

- For local search optimality and complete mean the same thing!
- No – can get stuck in local optima.



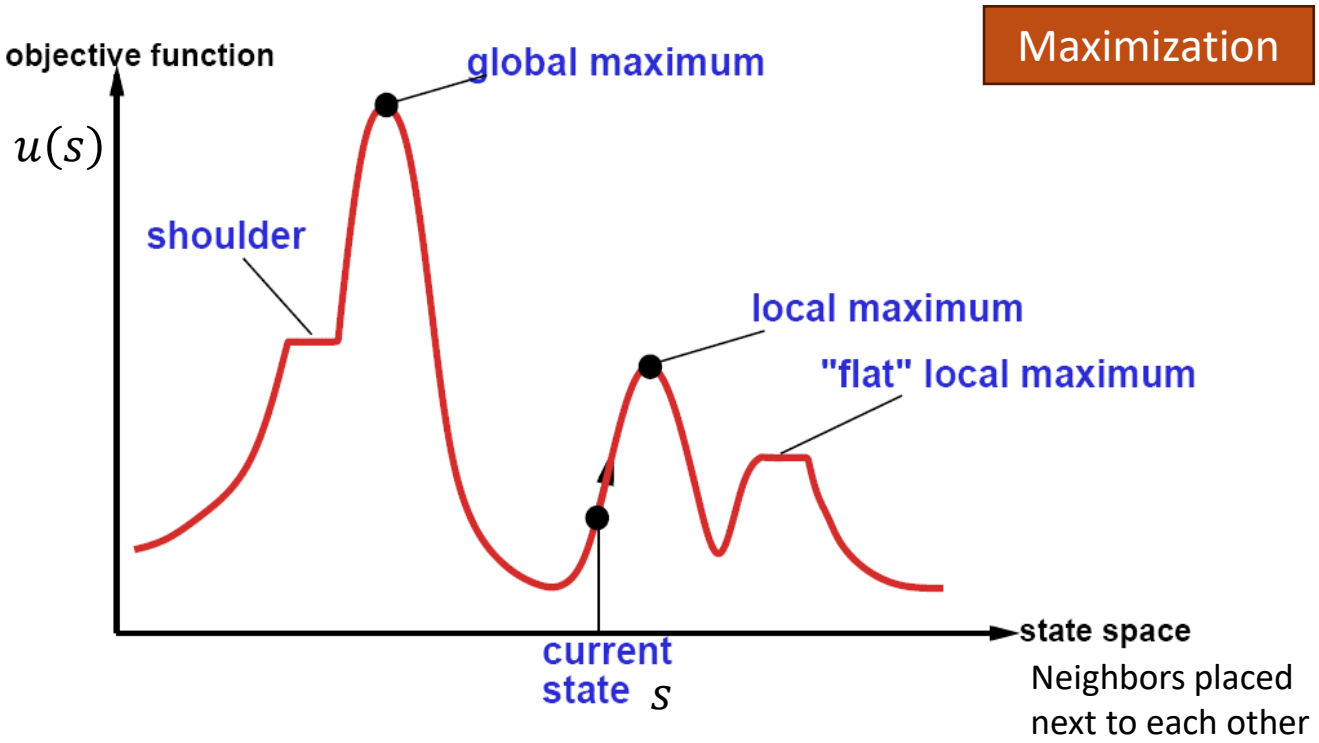
$h = 1$

Example: local optimum for the 8-queens problem. No single queen can be moved within its column to improve the objective function.

Simple approach that can help with local optima:

Random-restart hill climbing: Restart hill-climbing many times with random initial states and return the best solution. This strategy can be used for any stochastic (i.e., randomized) algorithm (randomized DFS, k-means, …).

The State Space "Landscape"



How to escape local maxima?

→ Random restart hill-climbing can help.

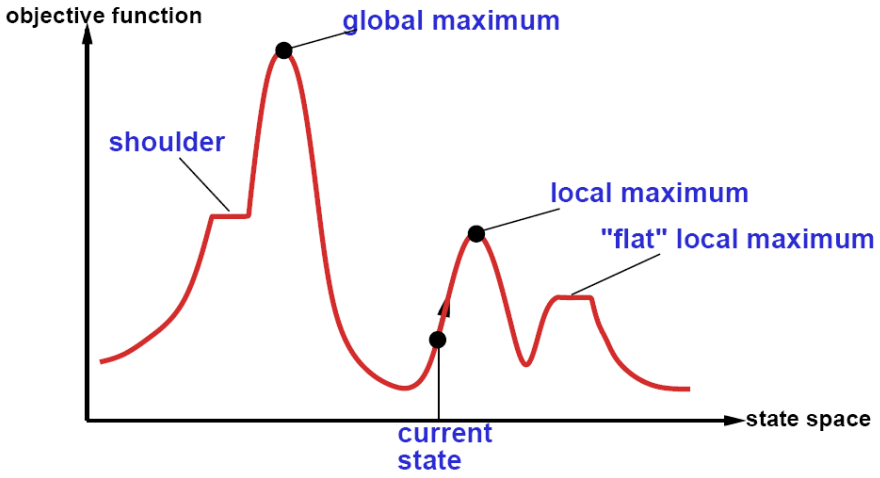
What about "shoulders" (called "ridges" in higher-dimensional spaces)?

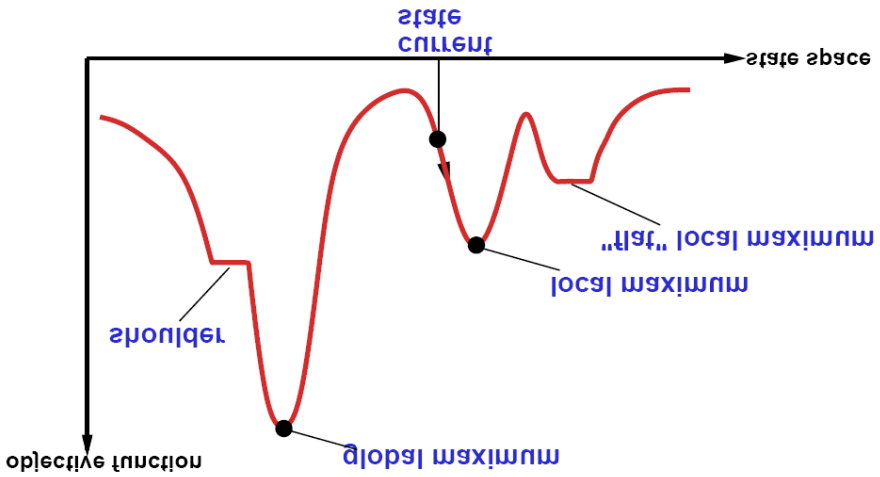
→ Hill-climbing that allows sideways moves and uses momentum.

Minimization vs. Maximization

- The name **hill climbing** used in AI implies **maximizing a utility function**.
- Optimizers often prefer to state problems as **minimization** problems to reduce **cost or loss** and refer to the same type of algorithm as **gradient descent**.
- Minimization and maximization are equivalent problems:

$$\max\big(f(x)\big) \quad \Leftrightarrow \quad \min\big(-f(x)\big)$$

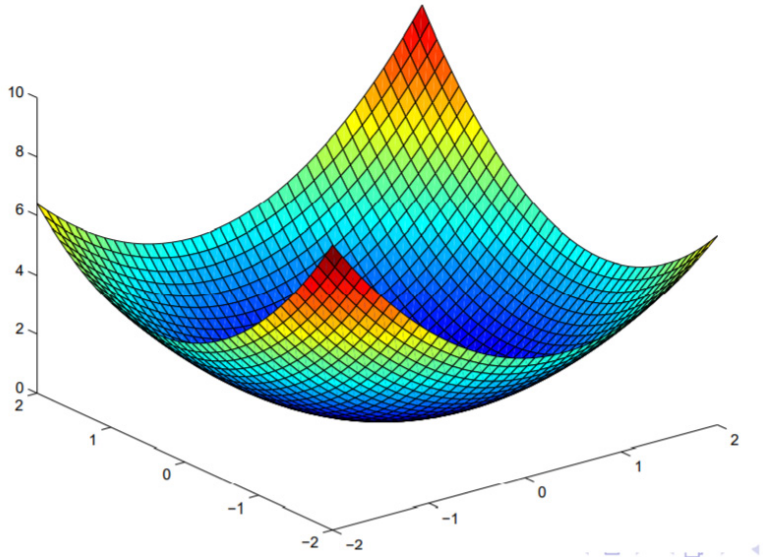




Convex vs. Non-Convex Optimization Problems

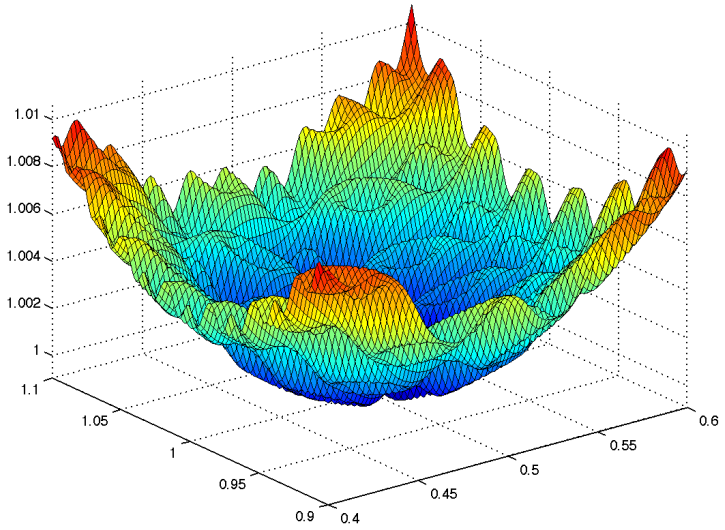
Minimization

Convex Problem



One global optimum +
continuous smooth function
→ calculus makes it easy
(solve $f'(x) = 0$)

Non-convex Problem



Many local optima → hard

Many AI problems are in addition discrete (the objective function is not differentiable). We often have to settle for a local optimum.

Code example: Apply Simple Hill Climbing to a Maze Problem

See Code on https://michael.hahsler.net/Introduction_to_Artificial_Intelligence/

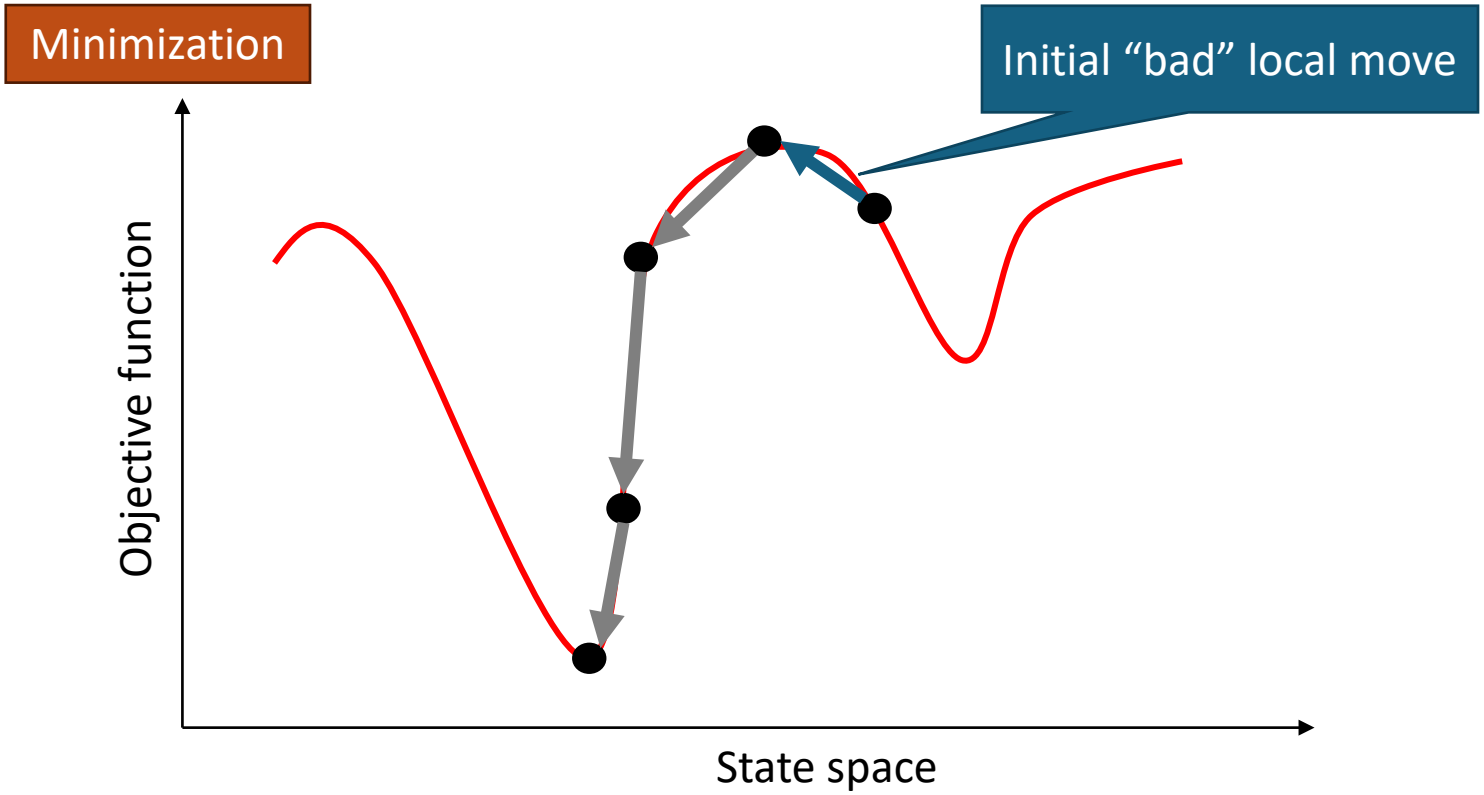


Simulated Annealing

Using heat to escape local optima…

Idea of Simulated Annealing

- Use first-choice stochastic hill climbing + escape local minima by **allowing some "bad" moves.**
- Gradually decrease the frequency of bad moves.
- Inspired by the process of controlled cooling of glass or metals. Decreasing the temperature means decreasing the chance of accepting bad moves.



Simulated Annealing Algorithm

- Use first-choice stochastic hill climbing + escape local minima by allowing some "bad" moves but gradually decreasing their frequency as we get closer to the solution.
- Annealing tries to reach a low energy state: A negative ΔE means the solution gets better.
- The probability of accepting "bad" moves follows the annealing schedule, which reduces the temperature T over time t.

Minimization

function SIMULATED-ANNEALING(*problem*, *schedule*) **returns** a solution state

current ← *problem*.INITIAL

Typically, we start with a random state

for $t = 1$ **to** ∞ **do**

$T \leftarrow schedule(t)$

if $T = 0$ **then return** *current*

next ← a randomly selected successor of *current*

$\Delta E \leftarrow VALUE(next) - VALUE(current)$

if $\Delta E \leq 0$ **then** *current* ← *next*

Always accept good moves that reduce the energy.

else *current* ← *next* only with probability $e^{-\Delta E/T}$

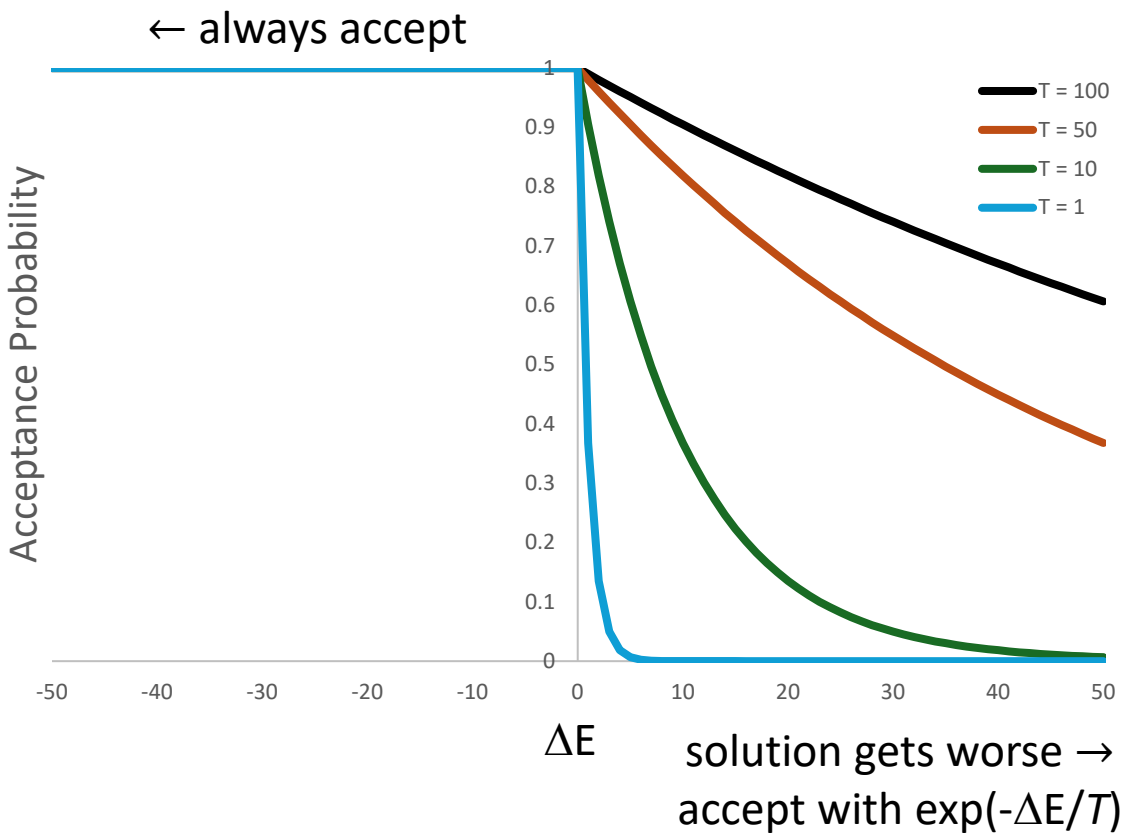
Accept "bad" moves using the Metropolis criterion (from MCMC algorithms for sampling from distributions).

Note: Use $VALUE(current) - VALUE(next)$ for maximization

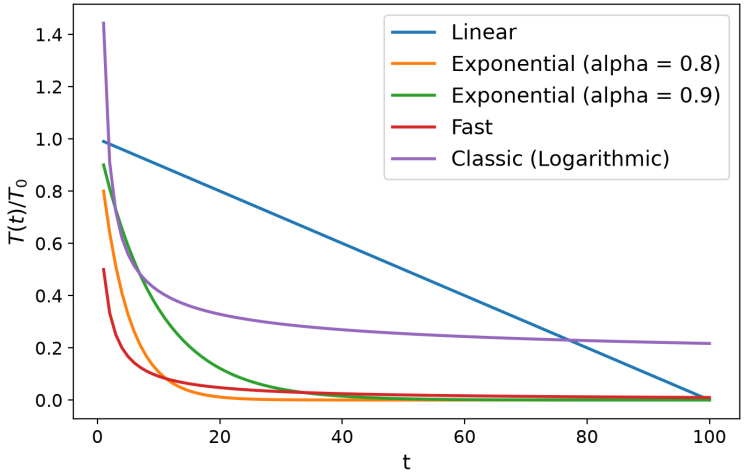
always accept

The Effect of Temperature

Convert the changes due to “bad” moves into an acceptance probability depending on the temperature. The criterion uses the negative part of the exponential function.



Cooling Schedule



The cooling schedule is very important.
Schedules for the temperature at time t:

- **Classic simulated annealing:** $T_t = T_0 \frac{1}{\log(1+t)}$
- **Exponential cooling** (Kirkpatrick, Gelatt and Vecchi; 1983)

$$T_t = T_0 \alpha^t \quad \text{for} \quad 0.8 < \alpha < 1$$

- **Fast simulated annealing** (Szy and Hartley; 1987)

$$T_t = T_0 \frac{1}{1+t}$$

This is the most popular choice!

Notes:

- Choose T_0 to provide a high probability $p_0 = e^{-\frac{\Delta E}{T_0}}$ that an average move will be accepted at time $t = 0$. For p_0 0.8 or 0.9 is often used. A conservative choice for ΔE is to use the worst possible move. This is a rule of thumb!
- Stopping rule: T_t will not become 0 but very small. Stop when $T < \epsilon$ (ϵ is a very small constant).
- The best schedule is typically determined by trial-and-error. The goal is to have a low chance of getting stuck in a local optima.

Simulated Annealing Search

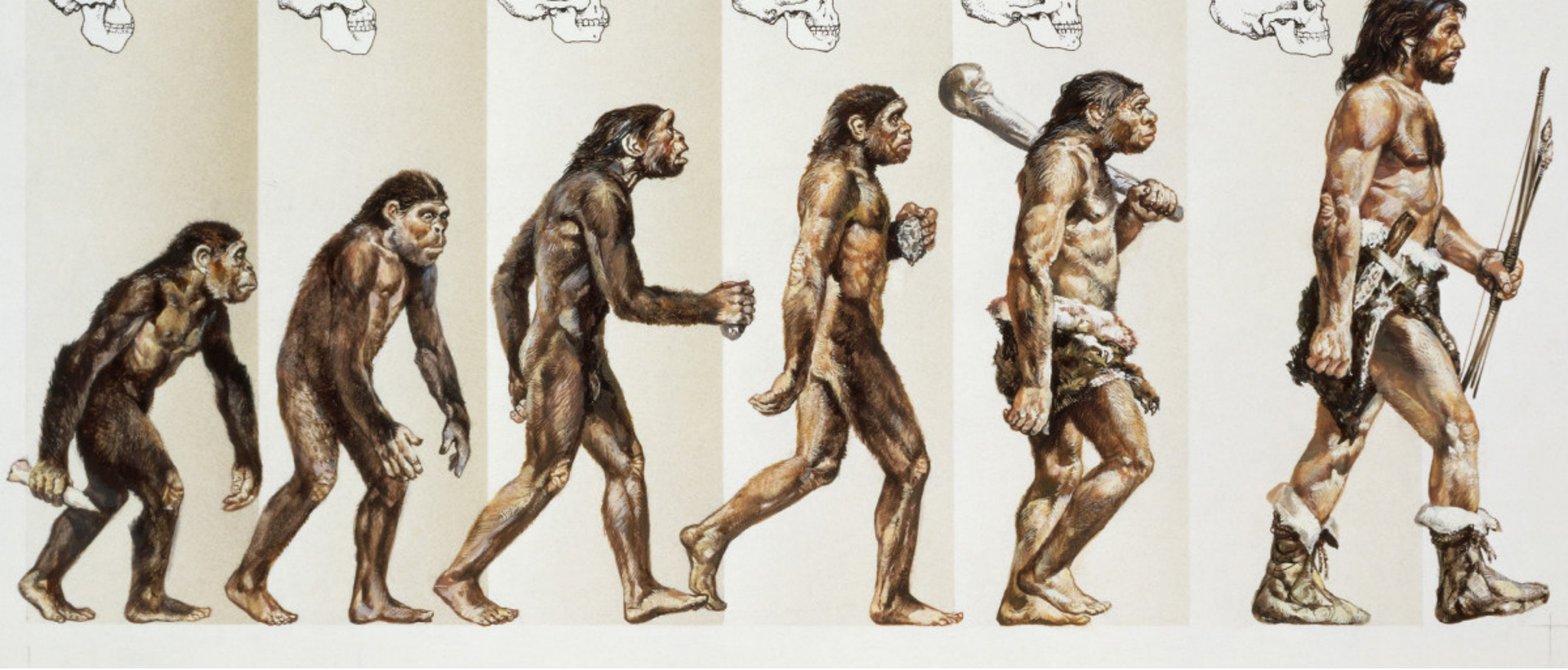
Guarantee

If the temperature is decreased **slowly enough** (using the classic cooling schedule), then simulated annealing search will find a **global optimum** with a probability approaching one.

However, this schedule takes too long and thus is **impractical**.

Practical solution

- Accept suboptimal results and determine the best cooling schedule and local move need experimentally.

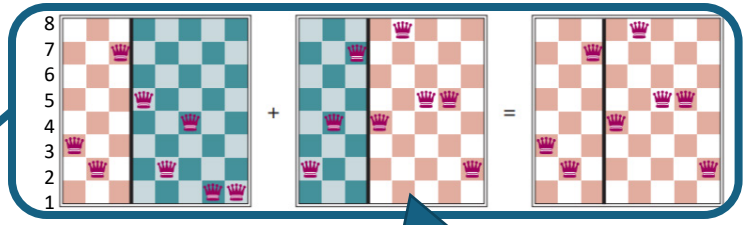


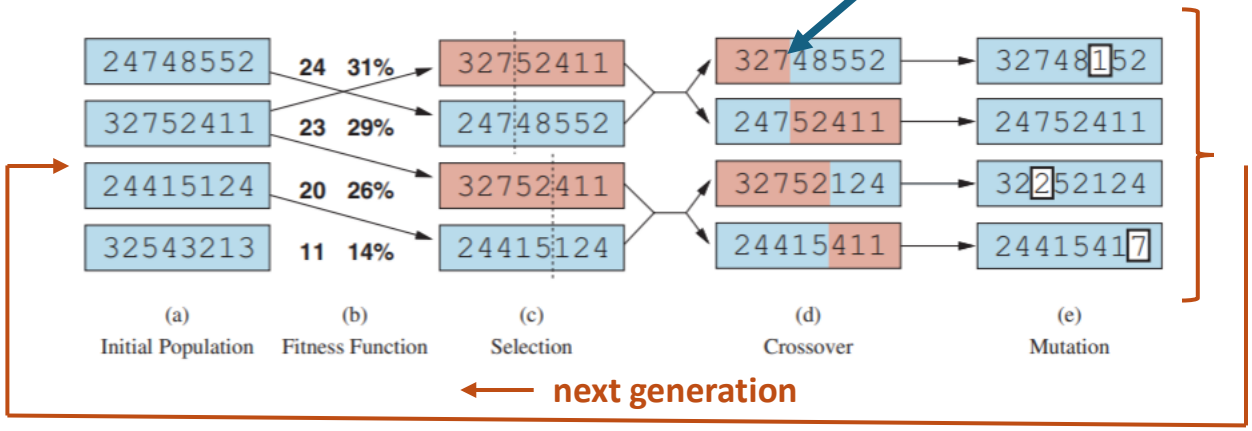
Evolutionary Algorithms

A Population-based Metaheuristics

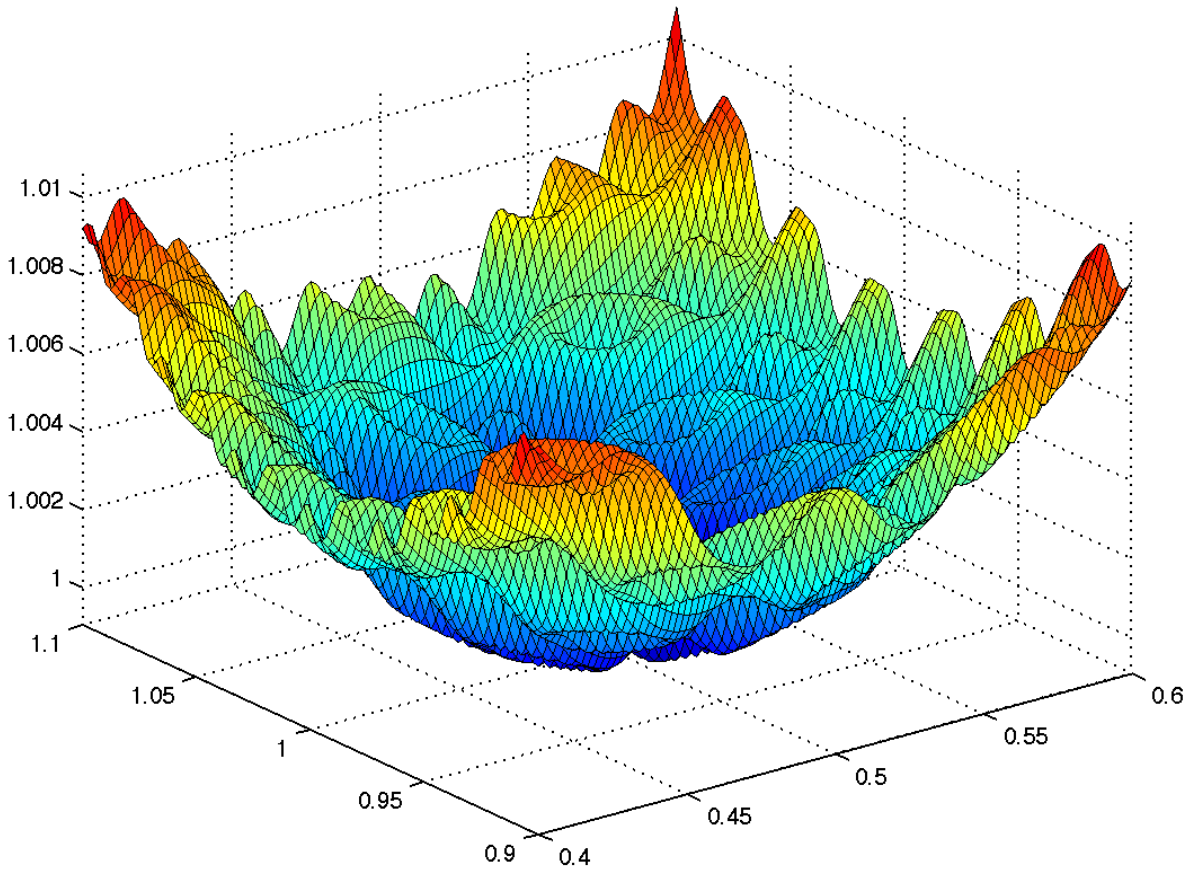
Evolutionary Algorithms / Genetic Algorithms

- A metaheuristic for **population-**based optimization.
- Uses mechanisms inspired by biological evolution (genetics):
  - Reproduction: Random selection with probability based on a **fitness** function.
  - Random recombination (crossover)
  - Random mutation
  - Repeated for many generations
- Example: 8-queens problem





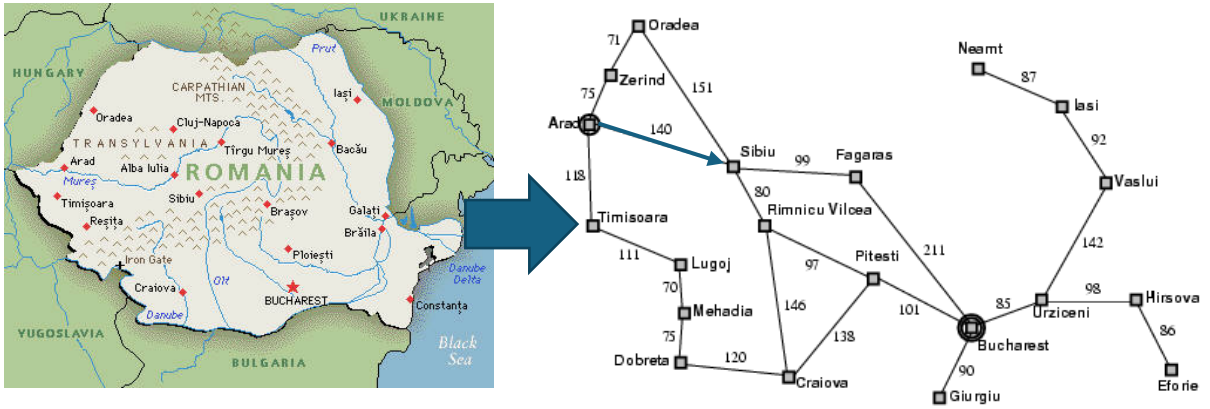
Individual = state **representation** as a chromosome: row of the queen in each column



Search in Continuous Spaces

Methods: Discretization of Continuous Space

- Use atomic states and create a graph as the transition function.



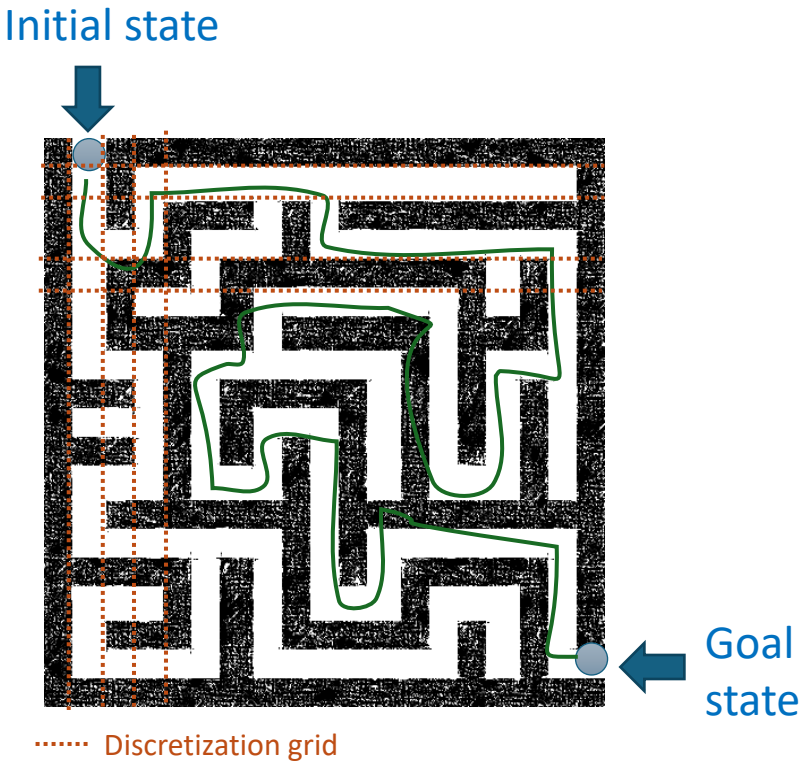
- Use a grid with spacing of size δ

  Note: You probably need a way finer grid!

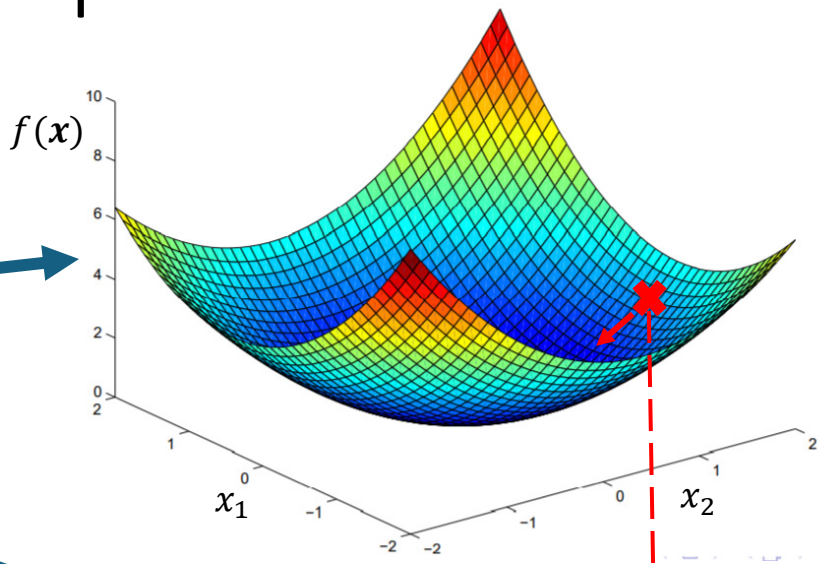


Example: Discretization of Continuous Space

How did we discretize this space?



Search in Continuous Spaces: Gradient Descent



State representation: $\boldsymbol{x} = (x_1, x_2, \ldots, x_k)$

State space size: infinite

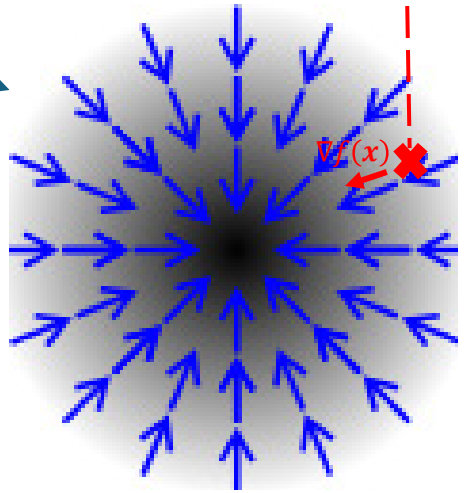
Objective function: min $f(\boldsymbol{x})$

Local neighborhood: small changes in $x_1, x_2, \ldots, x_k$

Gradient at point $\boldsymbol{x}$: $\nabla f(\boldsymbol{x}) = \left(\frac{\partial f(\boldsymbol{x})}{\partial x_1}, \frac{\partial f(\boldsymbol{x})}{\partial x_2}, \ldots, \frac{\partial f(\boldsymbol{x})}{\partial x_k}\right)$

(=evaluation of the Jacobian matrix at $\boldsymbol{x}$)

Find optimum by solving: $\nabla f(\boldsymbol{x}) = 0$



- **Gradient descent (= Steepest-ascend hill climbing for minimization)** with step size α (typically reduced over time)

  Repeat: $\boldsymbol{x} \leftarrow \boldsymbol{x} - \alpha \nabla f(\boldsymbol{x})$

- **Newton-Raphson method**
  uses the inverse of the Hessian matrix (second-order partial derivative of $f(\boldsymbol{x})$)
  $H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$ as the optimal step size

  Repeat: $\boldsymbol{x} \leftarrow \boldsymbol{x} - \boldsymbol{H}_f^{-1}(\boldsymbol{x}) \nabla f(\boldsymbol{x})$

Note: May get stuck in a local optimum if the search space is non-convex! Use simulated annealing, momentum or other methods to escape local optima.

Search in Continuous Spaces: Stochastic Gradient Descent

- **Issue**: What if the mathematical formulation of the objective function is not known? **We cannot find an equation for the gradient!**
- **Idea**: We may have objective function values at fixed points, called the **training data**. Can we approximate the gradient?
- **Remember**: Stochastic hill climbing uses random local moves. If the training data are a **random sample**, then we can use some of them to **approximate the gradient**. Approximating the gradient from sampled data is called **numerical or automatic differentiation**?

- **Algorithm**: The resulting algorithm is called **stochastic gradient descent (SGD).** It is called stochastic because it uses a random sample (the training data) to decide on the next local move.

→ We will discuss search in continuous spaces with loss functions using gradient descent when we discuss **parameter learning for learning from examples (machine learning).**



Conclusion

- Local search provides a fast method to find good solutions to many difficult optimization problems.
- Local optima are a big issue that can be addressed with random restarts and simulated annealing.